

什么是 Text-to-SQL

在各个企业数据量暴涨的现在，Text-to-SQL 越来越重要了，所以今天就来聊聊 Text-to-SQL。

Text-to-SQL 是一种将自然语言查询转换为数据库查询的技术。它可以让用户通过自然语言来查询数据库，而不需要编写复杂的 SQL 语句。

Text-to-SQL 的应用场景

我将 Text-to-SQL 的应用场景按照以下两个维度进行划分：

SQL 的复杂程度

生成 SQL 的准确性要求

SQL 的复杂程度

这里说的复杂除了是说 SQL 本身的复杂程度，比如要求生成的 SQL 语句中需要包含多个表的关联，多个子查询，或者需要包含复杂的聚合操作等。也体现在数据的复杂上面，数据库本身有大量的表，每个表有大量的字段，每个字段有大量的数据。这两点复杂性给 Text-to-SQL 带来了很大的挑战。

生成 SQL 的准确性要求

准确性则很好理解，就是能正确反映 Text 的内容，不要出错。

场景分类

我们期望的 Text-to-SQL 肯定是右上角这种，既能生成复杂的 SQL，又能保证准确性。但是目前来说，受限于知识工程的匮乏以及模型的能力，目前还没有一个完美的 Text-to-SQL 的解决方案。

左上角的区域是类似辅助 SQL 开发的场景，这种场景下，我们期望的是能生成一个 SQL 的草稿，然后我们再根据这个草稿进行修改，从而生成一个准确的 SQL。由于有 Human in the loop，所以这种场景下，Text-to-SQL 的准确性要求相对较低，但是生成 SQL 的复杂程度要求较高，需要能快速出一版大概能用的 SQL。

右下角的区域则是偏业务人员基于已经过滤得比较干净的数据，进行自助的报表查询、数据查询、数据分析的场景。由于业务人员没有读写 SQL 的技能，所以这种场景下，Text-to-SQL 的准确性要求较高，但是目前的技术没法做到同时保证生成 SQL 的复杂程度和准确性。所以只能退而求其次，让业务人员先使用右下角区域的 Text-to-SQL 了。

实现 Text-to-SQL

Text-to-SQL 的发展过程

Text-to-SQL 的发展过程可以分为以下几个阶段：

第一阶段：基于规则的 Text-to-SQL

第二阶段：基于深度学习模型的 Text-to-SQL

第三阶段：基于预训练模型的 Text-to-SQL

第四阶段：基于 LLM 的 Text-to-SQL

得益于 LLM 的强大语言理解能力和生成能力，Text-to-SQL 已经迎来了爆发式增长。通过下面的两个 Text-to-SQL 最著名的评测数据集，我们可以看到，基于 LLM 的 Text-to-SQL 已经完全碾压了所有其他的方法。

spider:

bird:

最简单的 Text-to-SQL

我们先来看一个最简单的 Text-to-SQL 的实现。

这个实现很简单，就是简单地把用户的查询和数据库中的 DDL 一起加到 Prompt 中，让 LLM 来生成一段 SQL。加入一点点 CoT 的技巧，这个实现在 Bird 上面可以达到 55%左右的执行准确率。现在排行榜上最好的结果是 77.1%左右，说明我们还有很大的提升空间。

执行准确率的意思是，生成的 SQL 语句在执行后能返回正确结果的比例。

Text-to-SQL 的挑战

想要提升 Text-to-SQL 的性能，我们得先分析一下 Text-to-SQL 的难点在哪里。基于我这段时间的调研，我将 Text-to-SQL 的难点总结为了以下 5 个挑战。每个挑战又对问题进行了拆解。

自然语言本身的复杂性

问题含义模糊

问题不完整

问题自带歧义

省略信息

代指信息

需要逻辑推理

错别字/同义词

缺乏业务上下文

业务术语理解

表/列的业务含义理解

数据库的复杂性

数据库类型多样性

大量的数据库表/列

严格的语法规则

模型有幻觉

模型不按指令生成 SQL

模型生成的 SQL 有瑕疵

模型生成的结果有随机性

很复杂的查询

join 大量的表

大量的嵌套查询

需要使用高级函数（窗口函数…）

依赖数据库特性

针对这些问题，我梳理了 8 大应对方案，最终将 8 大应对方案映射到了 3 类能力范围：模型能力、架构设计、知识工程，如下图所示：

下面来依次分析这些挑战。

挑战：自然语言本身的复杂性

问题含义模糊： 问题含义模糊是指用户的问题本身含义模糊，比如“我想知道南昌的销售额”，这个“南昌”是指“市”还是“县”，就需要澄清才能知道了。因为南昌既可以指“南昌市”，也可以指“南昌市”下面的“南昌县”。

问题不完整： 问题不完整是指用户的问题本身就不完整，比如“告诉我销售额是多少”，用户没有说清楚时间、地点等任何维度的销售额，也没说是总销售额，这就需要澄清才能知道了。

问题自带歧义： 问题自带歧义是指用户的问题本身就带有歧义，比如“查询购买了产品 A 和产品 B 的客户”。这句话是指查询“同时购买了产品 A 和产品 B 的客户”，还是指查询“购买了产品 A 或产品 B 的客户”，也需要澄清才能知道了。

应对这三个问题的方案是问题澄清，依赖于多轮对话的能力。最简单的方法就是把 Text-to-SQL 作为函数调用给到大语言模型，然后让大语言模型收集到足够且明确的信息后，再执行 Text-to-SQL。这么做的一个坏处就是，加重了 LLM 的“知识负载”，我尝试过让 gpt-4o 来进行这样的函数调用，效果不是很好，单独用一个 prompt 让 gpt-4o 来识别歧义的时候，它能识别出来；但是加上函数调用后，就没法识别了，直接就调用了 Text-to-SQL。所以现阶段如果想要效果做得好，还是得拆分任务。

省略信息： 省略信息是指用户的问题中省略了某些信息，比如用户先问了某产品的净利润，然后直接问“查询一下销售额”，用户没有说清楚是哪个产品的销售额，通过上下文可以知道是用户前面问的产品。

代指信息： 代指信息是指用户的问题中代指了某些信息，用户先问了“查询购买了产品 A 的客户”，然后又问“他们近一个月的总消费额是多少”，这里“他们”代指了用户前面问的客户。

应对这两个问题的方案是上下文的理解。其实 LLM 本身已经具备了这部分能力，唯一需要做的就是当上下文特别长的时候，我们可以通过总结的方式来减少上下文的长度，辅助 LLM 处理更长的上下文。

需要逻辑推理： 需要逻辑推理是指用户的问题需要进行逻辑推理才能得到最终的答案。推理本身也分了好多种，典型的推理类型有：演绎推理(Deductive Reasoning)，归纳推理(Inductive Reasoning)，溯因推理(Abductive Reasoning)，类比推理(Analogy Reasoning)。举个最简单的例子，模型的原本的知识里面知道身份证号码的前 6 位对应着地区，并且能映射上，如果让模型直接输出某个身份证号码的地区，它很可能输出一个错误的答案；但是如果让它先分解身份证前 6 位每 2 位和地区的对应关系，再输出地区，它就能输出对了。（这个还取决于模型，有的模型即使让推理也输出不了正确答案，因为该模型本身就没有“记住”数字和地区的对应关系）。

应对这个问题的方案是推理增强，具体的细节比较复杂，会在后面一节中单独讲解。

错别字/同义词： 错别字/同义词是指用户的问题中存在错别字或者同义词，比如“深圳的客户有多少个”，用户可能写成了“shenzhen 的客户有多少个”。

应对这个问题的方案是 Schema Linking，将用户的问题中的实体与数据库中的值进行关联。具体的应对方案比较复杂，会在后面一节中单独讲解。

挑战：缺乏业务上下文

业务术语理解： 用户的问题中可能包含一些特定的业务术语，而模型在训练的时候就没有注入这些业务术语的知识，属于企业内部的一些术语，那么模型对这些业务术语是没法理解的。

应对这个问题的方案是检索增强。我们需要梳理出所有问题中可能出现的业务术语，然后都用自然语言描述清楚术语具体是什么。最常见的就是别名了，可能用户说的是 Ax，然后 Ax 的别名是 A，数据库中存的也是 A，那么这条业务数据就很关键了。

实现方法则很简单（当然要做得比较好，需要花大量的时间）：混合检索一上，Rerank 一加，视场景复杂度也可以加上知识图谱检索。网上讲 RAG 的内容已经很多了，就不展开讲了。

表/列的业务含义理解： 企业内部的表列的业务含义是模型本身预训练的时候所没见过的，如果不将这些含义给到模型，生成 SQL 时用错表和列也就不奇怪了。

应对方案也是 Schema Linking，后面展开讲。

挑战：数据库的复杂性

大量的数据库的表/列： 一个企业内部的数据库中可能会存在大量的表、列，如果把所有的表和列信息都输入给模型，那它肯定就撑爆了。所以我们一定要有手段能够支持去筛选出和

本次用户问题相关的表、列。

应对方案有两个：一个是 **Schema Linking**，另一个是划分工作区。**Schema Linking** 会在后面展开讲，这里讲一下划分工作区的做法。

这个思想来自 **Uber** 的 **QueryGPT1**，我们需要给用户一个设置他自己的工作区的机会。这个思想很朴素：就是作为一个用户，我其实并不关心企业的整个数据库，我管好我一亩三分田就好了。所以用户可以自己划定工作区，只关注一部分表。某个用户可能身兼多职，那就按不同的职位划分不同的工作区就可以了。一个团队也可以划分团队的工作区。这样在生成 **SQL** 的时候，就只需要关注工作区中的表，大大降低了 **LLM** 的“知识负载”。

工作区的划分本身也不是一个很简单的事情，因为技术表和业务词汇中间可能还是有一些差距的，业务人员可能不知道如果选择表作为工作区，这个时候数仓建模的优势就体现出来了。业务域、数据域、业务过程的建模是高度业务化的，业务人员也能很容易得看懂，自然就更好选择自己工作区需要的表了。

严格的语法规则：**SQL** 有严格的语法规则，很小的错误也会导致整个流程失败，所以我们需要对执行时有语法错误的 **SQL** 进行修复，增强系统的鲁棒性。常见的小错误有：

单双引号混淆或忘加

表列名拼错

字段没按其类型操作

应对方案是 **Revise Agent**。

大语言模型其实是有修复这种错误的能力的，因为执行报错的时候数据库会给出错误信息，**LLM** 能根据错误信息进行 **SQL** 的修改，就跟我们人写复杂 **SQL** 的时候也不是一次写对，会根据语法错误来修改 **SQL** 是一样的。

所以可以引入一个 **agent**，把生成的 **SQL**、**SQL** 的执行结果或者报错信息都输入给模型，让模型去判断是否需要去修改 **SQL**，还是说现在的 **SQL** 就 **OK** 了，当然也需要去设置一下最大的重试次数。这样 **Agent** 就能不停地去修正 **SQL**，直到它觉得这个 **SQL** 是 **OK** 的了。

数据库类型多样性：一个企业，一般不会只使用一种类型的数据库。这就导致我们需要面对不同数据库的不同语法，比如 **MySQL** 允许单引号' 或双引号"包裹字符串，而 **Postgres** 必须用单引号，双引号是用于标识符（表名、列名）的。而模型有可能没有在对应的数据库语料中训练过，就导致生成的 **SQL** 会出问题。

应对方案有两种，一种是添加示例，另外一种就是微调模型。

添加示例：通过示例注入 **prompt** 的方式可以让模型学会方言的使用。但这种方式有一个问题：需要默认提供某数据库各式各样的大量示例，因为我们不知道用户会问什么问题，也不知道什么样的问题会用到什么样的数据库特性。

微调模型：微调涉及的内容范围也特别广，这里只提一下针对 **Text-to-SQL** 的任务，微调的大致方法。为了应对特定的数据库类型，所以我们肯定需要先让模型学会基本的语法，这个时候可以用大量的对应数据库的 **SQL** 来让模型先学会基本语法。最好是能通过继续预训练（**Continue Pretrain**）的方式来给模型注入语法知识。**SFT** 也可以，但是效果就没那么好了，而且数据集的准备也会更耗费精力一些。当模型学会基础语法后，就可以用多任务（1. 用户问题和 **SQL** 回复；2. 通过改写问题扩充训练数据；3. 给定 **SQL** 让生成问题；4. 修复有错误的 **SQL**；5...）加上通用的任务来进行 **SFT** 训练。下面是 **Bird** 榜单上得分 **75.63** 的 **XiYan-SQL2** 的模型微调流程：

挑战：模型有幻觉

模型生成的结果有随机性： 相信大家在平时做 **LLM** 应用的时候都遇到过这个问题，即使我们将温度设置为 **0**，**Top P** 设为 **0**，模型的输出还是带有随机的，能影响结果的因素实在是太多了。

模型忽略了指令： 模型没有遵循全部的指令，比如我们要求模型添加一些默认字段，模型却只添加了一部分，漏了一些默认字段。

模型生成的 **SQL** 有瑕疵： 比如列名或表名抄错了。

应对这三个问题的方案都是模型微调和推理增强。

挑战：很复杂的查询

依赖数据库的 **Dialect**： 某些查询可能需要依赖数据库的方言，比如 **pivot** 功能的实现，在 **Oracle** 和 **Postgres** 里就不一样。

需要使用高级函数： 某些查询得用到窗口函数这种比较复杂的操作

join 大量的表： 有些查询会 **join** 很多张表

大量的嵌套查询： 有些查询涉及到多层嵌套查询

应对这些问题的方案都是模型微调和添加示例。

我这边想提的一点是，虽然上面的问题可以通过这两个方案来解决，但是目前来看也只能解决一部分，模型的能力还是没有那么强，新出的 **BIRD-CRITIC** 榜单上，最强的模型目前也才 **38.5%** 的准确性。

所以我们需要通过工程手段，来尽量避免掉这些复杂性。**join** 太多？那就看看是不是某些表是可以合并的，用数仓的维度建模可以降低 **join** 的复杂性。某些计算逻辑很复杂？那就看看是不是能提前计算一下。

应对方案

应对方案：推理增强

回顾一下推理增强能解决的问题：

自然语言的复杂性：

需要逻辑推理

模型有幻觉：

模型生成的结果有随机性

模型忽略了指令

模型生成的 SQL 有瑕疵

大模型本身具备了一定的推理能力，我们要做的就是引导出模型进行推理即可。加上了 CoT（Chain of Thought）之后，推理能力就能显著提升。这里我用自己做得一个微调小实验来证明这一点。

我做的微调 实验是关于英雄联盟的，所以需要先给大家一个背景知识，英雄联盟是一款游戏，游戏中有很多英雄，而每个英雄都有被动技能。英雄除了比较官方的名字之外，我们在玩的时候都会喊他的外号：比如官方名为“复仇焰魂·布兰德”的英雄，我们一般会叫他“火男”。

我先测试了一下 XiYanSQL-QwenCoder-32B2 对于火男这个概念是否了解，结果如下：

Q:英雄联盟中火男叫什么，全称是什么

A:英雄联盟中的火男（莫甘娜）的被动技能名为“燃烧之魂”。这个技能的效果是，莫甘娜的技能会对敌人造成额外的魔法伤害，并且在敌人死亡时，她会恢复一定量的生命值。这个被动技能使得莫甘娜在团战中能够持续输出伤害并保持自身的生存能力。

AI 写代码

1

2

模型对于火男这个概念存在幻觉，认为他是莫甘娜了。

然后我就通过 lora 的微调，给模型添加了“火男相关”的知识，然后再次测试，结果如下：

Q:英雄联盟中的火男的被动是什么。

A:火男的被动技能是“炽热之焰”。

AI 写代码

1

2

然后我们让其写 SQL，用火男的被动技能名称来查询技能描述，这需要模型知道火男的被动技能是“炽热之焰”，然后把“炽热之焰”作为查询条件。

```
Q:CREATE TABLE skill_description (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  description TEXT NOT NULL  
);
```

根据 ddl 生成 sql: 帮我查一下英雄联盟火男的被动技能的描述, 通过技能的名字来查

A:SELECT description FROM skill_description WHERE name = '火男的被动技能';

AI 写代码

1
2
3
4
5
6
7
8

可以看到, 模型在填写 name 的时候, 没有足够的背景知识知道这里应该填写“炽热之焰”, 这是因为我的训练语料中, 只说了火男的被动技能是“炽热之焰”, 但是这种转 SQL 的场景并没有特别交代, 而且训练的数据量也不大, 没法改变模型的固有思维。

那就再限制一下模型, 只允许他填写真实的技能名字:

Q:...

根据 ddl 生成 sql: 帮我查一下英雄联盟火男的被动技能的描述, 通过技能的真实名字来查, 别用占位符

A:SELECT description FROM skill_description WHERE name = '火之意志';

AI 写代码

1
2
3
4

结果还是不对, 在转 SQL 的数据分布下, 模型还是不知道要填写“炽热之焰”。

我们再来试试看 CoT, 模型已经能知道火男的被动技能是“炽热之焰”了, 如果他在输出 SQL 前先把这个知识给补全, 那是不是就能填对了。

Q:...

根据 ddl 生成 sql: 帮我查一下英雄联盟火男的被动技能的描述, 你先思考一下火男被动的名字是什么, 然后通过技能的名字来查。

A:在英雄联盟中, 火男的被动技能名称是“炽热之焰”。因此, 我们可以使用这个名称来查询技能描述表中的描述信息。

SELECT description FROM skill_description WHERE name = '炽热之焰';

AI 写代码

1
2
3
4

5

6

这次模型就能正确地填对了，这个小实验证明了 CoT 在推理能力上的有效性。如今大火的模型 DeepSeek-R1 其实也是基于这个最基本的思想进行的。让模型能在生成的时候更多地思考，从而提升推理能力；不同是 R1 模型是通过训练模型的方式，并且从结果层面进行了正确答案的对齐，确保思考的正确性概率更大，最终效果更好。

既然提到了 CoT，那不得不提一下 Self Consistency 这个方法。这个方法的思路是：通过给模型设置较高的 Temperature，让模型在生成的时候有更多的随机性，从而生成更多的可能的答案，然后从这些答案中选择结果数量最多的答案。这个方法经过大量的实践，在多个数据集上都能显著提升效果。在 Text-to-SQL 领域自然也是适用的。这里的门道就是，模型本身是有随机性的，如果它只有 80%，而不是 100% 的概率能生成出一个对的 SQL，那么，为了每次都让他能生成正确的 SQL，我们让他生成 10 次，即使偶尔错了 2/3 次，那还是有 7/8 次是正确的，最终投票选出来的，就一定会是一个正确的 SQL 了。看一下 Spider 数据集的排行榜，可以发现第二名就用了这个方法。

直接使用 Self Consistency 还是会出问题的。在 Bird 数据集上，我们就不太见到 Self Consistency 的方法了，原因其实有以下两点：

结果数量最多，并不完全意味着答案更准确。
设置比较高的温度会导致生成 SQL 的准确率下降。
所以针对这两点，都有了相应的改进策略。

既然结果不是多数胜出，那就专门用一个模型来选择答案就行了。Bird 排行榜上的前两名都用了这个方法。选择答案的模型可以用 Prompt 来实现，也可以训练一个单独的模型来实现；要想效果更好，还是得基于 LLM 训练一个单独的模型。CHASE SQL3 显示了训练模型的效果提升，一个 9B 的模型，也能击败 Claude-3.5-sonnet 和 Gemini-1.5-Pro：

既然较高的温度会导致生成 SQL 的准确率下降，那么想办法生成高质量的候选 SQL 就行了，这个思路同样来自 CHASE SQL3。我们可以采取以下三种方式来生成高质量的 SQL：

Divide and Conquer CoT（分而治之 CoT）：让模型把查询任务拆解成更细粒度的任务，然后用 SQL 伪代码写出对这些子任务的查询，最终将结果合成一个完整的 SQL。这个想法很自然，直接写 SQL 效果不好是因为模型可能没有见过这样的数据分布（一个复杂的问题对应到一个复杂的查询），但是它肯定见过简单任务的数据分布，在 SQL 子语句都写出来了的前提下，SQL 的组合就显得没那么复杂了。

Query Plan CoT（查询/执行计划 CoT）：让模型先把 SQL 的执行计划描述出来，再通过执行计

划来生成 SQL。这么做可以让模型换个角度思考问题，将数据分布切换到了另外一个领域，能让模型更多地关注具体要用的表、列，能补足“分而治之”方法对细节把控不足的缺点。

Online Synthetic Example Generation（在线示例合成）：给示例的前提是多样性，因为如果多样性不足，很可能 LLM 就参考着示例写错了。所以示例也可以让 LLM 来生成，我们提供的是生成规则和生成数据来源，比如让它生成带/不带 join，带/不带聚合，带/不带高级函数等等的示例。也可以让它根据所有表列或者 Schema Linking 后的表列来生成示例。

应对方案：Schema Linking

回顾一下 Schema Linking 能解决的问题：

自然语言复杂性：

错别字/同义词

数据库的复杂性：

大量的数据库表/列

缺乏业务上下文：

表/列的业务含义理解

先讲一下 Schema Linking 是什么吧？看一下上面这个例子，将自然语言中的实体、关键词和数据库中的表、列、值关联起来，就是 Schema Linking。

这个例子里，针对“查询在 shenzhen 购买了产品 A 的客户 id”这个输入，shenzhen 会 link 到数据库 location 表中的深圳市的值，购买则会去 link 到 orders 表，然后产品 A 的话会 link 到 product 表里面的 A product 的值。客户 id 则会 link 到 orders 表的 customer_id 列，注意，link 到最底层的值的时候，我们同时会把它的列以及它的表也都给取回，丰富上下文信息。最后还需要关注的是表与表之间的 link，这个现在通常的做法就是默认把主键和外键都带上。

针对上面提到的三个问题，Schema Linking 是这么来解决的：

错别字、同义词的纠正：

比如说在这个例子上面，用户查询中，深圳其实就用了一个拼音 shenzhen，而数据库中存的是深圳市。

对于这种情况的，如果不去匹配数据库里具体的值，那么模型就会用 shenzhen 的拼音去做 where 的过滤条件了，所以需要 will 将用户查询中的错别字同义词，link 到表里面的具体值。

数据库里面有大量的表和大量的列：

客户 id 需要被映射到 order 表的 customer_id 列，购买需要被映射到 orders 表。

我们需要对这些表和列的进行筛选过滤，也就是说我们需要通过用户的查询输入，找到跟用户查询最相关的表和列，然后只把这些表和列输入给模型，让它去生成 SQL，不然的话大量的表和列信息太多，甚至都没办法输入到模型里面去，即使能都放进去，同时输入的 token 数量过多，模型的性能是会显著下降的。

表/列的业务含义理解：

最后就是表/列可能隐含了一些特殊的业务知识，而模型对与这些知识是不清楚的，那需要去取到这些表/列对应描述，帮助模型来理解表和列的具体含义。不把所有的表描述和列描述都放到 prompt 中，还是因为放太多会导致模型的性能下降。

具体实现细节

我们来详细展开讲一下实现。

错别字/同义词的问题

首先是值的 link 问题：目前主流的做法是使用文本匹配和语义相似性。文本匹配更适合那些没有语义信息的数据，比如名字、电话、域名等。而语义相似度适配的范围就很广了，性别、职业、爱好等，都可以使用语义相似度来匹配。

值得一提的是文本匹配主要还是英文的做法，因为英文中的字母错拼的概率更大，一个单词中出现一两个字母的错误，其实还是很容易匹配的。但是中文环境下，其实错误更多发生在两种情况（注：暂时不考虑使用五笔输入法的小部分情况），一个是同音字，一个是拼错了键盘邻近的字母，原本可能就 2、3 个字的内容，直接按中文做匹配，召回率会非常低。所以中文不太能直接用字来进行数据库值的查找，转而使用拼音匹配能增大召回率。具体的实现方法的话，可以参考 CHASE SQL3 中的做法。先对拼音进行 ngram 划分，再使用 MinHash+LSH 来召回一批结果，最后通过编辑距离来进行更精确地筛选。

而语义相似性的话大家应该就很熟悉了，选个好用的 Embedding 模型（中文推荐 xiaobu-embedding-v2，好用，成本也低），然后通过向量数据库来存储和检索，最后用 rerank 模型来精排一遍，选出最合适的值。

这里值的召回还需要提一点，就是值都是会和具体的表和列相关的，所以在查询的时候有两种查询方式，一种是直接查询值，一种是加上表和列再来查询值，具体要在什么时候用什么方法，就需要看用户的输入信息了。如果用户的输入信息中包含了列的信息，那么，同时用列加值查询效果就会更好。比如用户的输入是“查询业务类型是 A 的产品”，那么我们在知道列是业务类型的情况下，再从业务类型的列中进行值的召回，效果就会更好。如果要通过表和列进行过滤，就需要注意向量数据库索引类型的选择，向量数据库正在往向量索引和标量索引两类索引发展了，标量索引用于先过滤缩小筛选范围，向量索引用于向量匹配。

另外一个要解决的就是，拿什么去做检索和匹配，我们不能直接拿着用户的问题去检索和匹配，因为用户的问题中往往包含太多的信息，我们想要的是用户的问题中包含的一些关键词，比如 shenzhen，A Product，客户 id，这些关键词才是我们想要去匹配的。关键词的提取可

以交给 LLM，现在 LLM 对于提取关键词已经很厉害了，只需要给个 few-shot 的 prompt，就能控制模型提取关键词的方式了。我之前在一个场景下，需要 LLM 给我们提取具体的量词，比如当用户问“xxx 有几户”的时候，这个几户会需要进行相应的检索，一开始模型是不会提取几户作为关键词的，后面我在 few-shot 中加了一个示例，模型就能很好地提取出几户作为关键词了。

大量的数据库表/列

值选择完之后就可以开始选列了，选列主要有几步：

根据值选择的结果，先获取到对应的列

根据问题和关键词去进行列名和列描述的语义相似度的召回（一般列名都是有语义的）

将值选择的列和召回的列合起来，让 LLM 来选择最合适的列。（可以筛选掉比较多不相关的列）

添加上一些默认字段，比如主键、外键等

选表也是一样的逻辑：

根据值选择的结果，先获取到对应的表

根据问题和关键词去进行表名和表描述的语义相似度的召回

将值选择的表和召回的表合起来，让 LLM 来选择最合适的表。

表/列的业务含义理解

这个的话，还是通过检索增强来实现。值得一提的是，某些特定的计算逻辑可以直接在描述里写出 SQL 来帮助模型减负，比如某个状态需要通过判断枚举值等于 A 或 B，那就直接用 SQL 写出来：CASE WHEN ... THEN 1 Else 0 as XXX 状态。当然，这种计算最好是通过工程手段直接用 ETL 处理好！

总览

将上面提到的内容合起来，就可以得到下面这张架构图：

如果能把这里面所有的内容都实现了的话，针对具体场景做到 90%以上的执行准确率是没有问题的，有问题尽管来找我。

版权声明：本文为 CSDN 博主「boydfd」的原创文章，遵循 CC 4.0 BY-SA 版权协议，转载请附上原文出处链接及本声明。

原文链接：<https://blog.csdn.net/boydfd/article/details/146605141>